

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a8adad7ff12312445.jsonl`  
- SHA-256: `43f9868971310082c56b364b00cee1460f8fb49da2e74c707c04af2beb1883a7`  
- Source modified: `2026-06-10T16:28:02+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:25:50.933Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/gmail-attachments-to-gdrive and reported this finding:

```
TITLE: Incremental sync advances historyId past messages that were not fully processed (silent data loss)  
SEVERITY CLAIMED: high  
FILE: src/sync.ts (line 175-189)  
DESCRIPTION: In the incremental branch, state.setHistoryId(changes.historyId) is executed unconditionally after updateDrive returns. updateDrive deliberately does NOT mark a message processed when the daily cap truncates it mid-message or when an attachment copy throws (drive.ts:174-180), with the stated intent that 'the next run retries it'. But in incremental mode the only retry mechanism is the history cursor, and it is advanced past those messages regardless. The next run lists only changes AFTER changes.historyId, so cap-truncated and transiently-failed messages are never seen again ? their attachments are silently never copied. There is no fallback re-scan to catch them (the 30-day rewind only happens on cursor expiry). The feature is off by default (ENABLE_INCREMENTAL_SYNC=false), but this is the designed steady state once enabled.  
EVIDENCE QUOTED: // Advance the cursor so the next run only sees newer changes.  
    state.setHistoryId(changes.historyId);  
...drive.ts: // If we stopped early on the daily cap (lastHandled short of the end) or  
// hit a transient failure, leave it unmarked so the next run retries it.  
SUGGESTED FIX: Only advance the cursor when the run completed cleanly: if updateDrive reported errors or stopped at the cap (addedFiles == maxToAdd with messages left), keep the old historyId (re-listing is cheap and dedup via processedSet + the filename probe makes overlap safe). Alternatively, persist the unprocessed message ids in a small retry queue before advancing.
```

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
 - If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
 - Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
 - Do not use WebSearch/WebFetch and do not send any repo content to any external service.
- Structured output only.

2. user (2026-06-10T16:26:00.589Z)

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
```

3. user (2026-06-10T16:26:00.738Z)

```
1      import {
2          ENABLE_INCREMENTAL_SYNC,
3          MAX_FILES_ADD_PER_DAY,
4          MAX_RUNTIME_MS,
5          MAX_SYNC_TRIGGERS,
6          ONE_DAY,
7          ONE_MINUTE,
8          SIX_HOURS,
9          THIRTY_DAYS,
10     } from "./config";
11     import { updateDrive, type UpdateResult } from "./drive";
12     import {
13         getCurrentHistoryId,
14         listAddedSince,
15         threadsForMessageIds,
16     } from "./gmailHistory";
17     import { logDebug, logError } from "./logger";
18     import { maybeSendDailySummary } from "./notify";
19     import { addProcessedId } from "./processed";
20     import { buildSearchQuery } from "./query";
21     import { hasTime, isSameCalendarDay } from "./quota";
22     import { dayKey, recordRun } from "./stats";
23     import * as state from "./state";
24
25     const SYNC_HANDLER = "syncUserDrives";
26
27     /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
28     export function newSyncTrigger(afterMs: number): void {
```

```

29     ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
30 }
31
32 /** All currently-installed triggers for the sync handler. */
33 function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
34     return ScriptApp.getProjectTriggers().filter(
35         (t) => t.getHandlerFunction() === SYNC_HANDLER,
36     );
37 }
38
39 /** Removes every project trigger (used when the user re-runs setup). */
40 export function deleteAllTriggers(): void {
41     for (const trigger of ScriptApp.getProjectTriggers()) {
42         ScriptApp.deleteTrigger(trigger);
43     }
44 }
45
46 function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
47     for (const trigger of triggers) {
48         ScriptApp.deleteTrigger(trigger);
49     }
50 }
51
52 /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
53 function pruneSyncTriggers(max: number): void {
54     const triggers = getSyncTriggers();
55     for (let i = max; i < triggers.length; i++) {
56         ScriptApp.deleteTrigger(triggers[i]);
57     }
58 }
59
60 /**
61  * The background worker. A per-user lock ensures only one run executes at a time
62  * for THIS user, so concurrent triggers cannot stack file copies or trigger
63  * generations. It is deliberately a user lock, not a script lock: the web app
64  * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
65  * would serialise every user's sync into one global queue.
66  */
67 export function syncUserDrives(): void {
68     const lock = LockService.getUserLock();
69     if (!lock.tryLock(0)) {
70         logDebug("Another sync run is in progress; skipping.");
71         return;
72     }
73     try {
74         runSyncOnce();
75     } finally {
76         // Bound the trigger count regardless of which path we took, then release.
77         pruneSyncTriggers(MAX_SYNC_TRIGGERS);
78         lock.releaseLock();
79     }
80 }
81
82 /**
83  * A single sync pass:
84  * 1. reschedules a fan-out of backup triggers (self-healing if one is lost),
85  * 2. syncs forward in 30-day windows until it runs out of time, hits the
86  *    daily file cap, or fully catches up,
87  * 3. persists progress and schedules the next run.
88  */
89 function runSyncOnce(): void {
90     const today = new Date();
91
92     // On the first run of a new calendar day, summarise the previous day.
93     maybeSendDailySummary(today);

```

```

94
95 // Recover state if the debug flag was cleared (e.g. properties were wiped).
96 const debug = state.getDebug();
97 if (debug == null || debug === "0") {
98     state.setDebug("1");
99     state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
100 }
101
102 // Create the backup triggers BEFORE deleting the current ones, so a failure
103 // here can never leave us with no scheduled run.
104 const previousTriggers = getSyncTriggers();
105 try {
106     newSyncTrigger(60 * ONE_MINUTE);
107     newSyncTrigger(SIX_HOURS);
108     newSyncTrigger(2 * SIX_HOURS);
109     newSyncTrigger(4 * SIX_HOURS);
110     newSyncTrigger(8 * SIX_HOURS);
111     newSyncTrigger(28 * SIX_HOURS);
112 } catch (e) {
113     // Could not install new triggers; leave the old ones and try again later.
114     logError("Could not install triggers: " + e);
115     return;
116 }
117 deleteTriggers(previousTriggers);
118
119 const filesAddedToday = state.GetFilesAddedToday();
120 let added = filesAddedToday.added;
121 const lastCountDate = new Date(Date.parse(filesAddedToday.date));
122
123 if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
124     logDebug("Used up all quota for today. Can't add more files.");
125     newSyncTrigger(60 * ONE_MINUTE);
126     return;
127 }
128 if (!isSameCalendarDay(lastCountDate, today)) {
129     added = 0;
130 }
131
132 const structure = state.getFolderStructure();
133 let lastSynced = state.getLastSynced();
134
135 // Load the dedup set once; thread it through the run and persist at the end.
136 let processedIds = state.getProcessedIds();
137 const processedSet = new Set(processedIds);
138
139 // This run's contribution to the day's stats.
140 const addedAtStart = added;
141 let runErrors = 0;
142 let runLastError = "";
143
144 // Folds one updateDrive result into this run's accumulators.
145 const applyResult = (result: UpdateResult): void => {
146     added += result.addedFiles;
147     runErrors += result.errors;
148     if (result.lastError) {
149         runLastError = result.lastError;
150     }
151     for (const id of result.processedIds) {
152         if (!processedSet.has(id)) {
153             processedSet.add(id);
154             processedIds = addProcessedId(processedIds, id);
155         }
156     }
157 };
158

```

```

159 try {
160     // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
161     // gradually over the proven date-window backfill. A stored historyId only
162     // exists once a backfill has caught up while the flag was on.
163     const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
164     if (historyId) {
165         // Incremental: process only messages added since the stored historyId.
166         const changes = listAddedSince(historyId);
167         if (changes == null) {
168             // History expired: clear the cursor and re-scan a bounded recent window
169             // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
170             // dedup by message id and the Drive filename guard prevent re-copying.
171             state.clearHistoryId();
172             lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
173             logDebug("History expired; reverting to date-window sync.");
174         } else {
175             if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
176                 const threads = threadsForMessageIds(changes.messageIds);
177                 if (threads.length > 0) {
178                     applyResult(
179                         updateDrive(
180                             threads,
181                             structure,
182                             MAX_FILES_ADD_PER_DAY - added,
183                             processedSet,
184                         ),
185                     );
186                 }
187             }
188             // Advance the cursor so the next run only sees newer changes.
189             state.setHistoryId(changes.historyId);
190         }
191     } else {
192         // Backfill: walk forward in 30-day windows until caught up to today.
193         // Capture the mailbox historyId BEFORE processing so mail arriving
194         // mid-run is still picked up by the first incremental scan.
195         const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
196             ? getCurrentHistoryId()
197             : null;
198         let fullSynced = false;
199         while (
200             hasTime(new Date(), today, MAX_RUNTIME_MS) &&
201             added < MAX_FILES_ADD_PER_DAY &&
202             !fullSynced
203         ) {
204             const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
205             const search = buildSearchQuery(lastSynced, new Date());
206             logDebug("Query: " + search.query + " files asking: " + maxToAdd);
207
208             const threads = GmailApp.search(search.query);
209             if (threads.length > 0) {
210                 applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
211             }
212
213             fullSynced = search.isToday;
214             if (added < MAX_FILES_ADD_PER_DAY) {
215                 lastSynced = fullSynced
216                     ? today.toJSON()
217                     : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
218             }
219         }
220         // Caught up: switch future runs to the cheaper incremental path.
221         if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
222             state.setHistoryId(candidateHistoryId);
223             logDebug("Backfill complete; stored historyId " + candidateHistoryId);

```

```

224     }
225   }
226   } catch (e) {
227     logError("Some error in execution: " + e);
228     state.setLastError(String(e));
229     runErrors++;
230     runLastError = String(e);
231   }
232
233   // Fold this run into the day's accumulator.
234   const dayStats = recordRun(state.getRunStats(dayKey(today)), {
235     filesAdded: added - addedAtStart,
236     errors: runErrors,
237     lastError: runLastError,
238   });
239   state.setRunStats(dayStats);
240
241   state.setProcessedIds(processedIds);
242   state.setLastSynced(lastSynced);
243   state.setFilesAddedToday({ added, date: today.toJSON() });
244   newSyncTrigger(20 * ONE_MINUTE);
245 }
246

```

4. user (2026-06-10T16:26:07.863Z)

```

1   import { GMAIL_ATTACHMENT_FOLDER } from "./config";
2   import { isAllowedContentType } from "./contentTypes";
3   import { getAttachmentName, getFolderNameForStructure } from "./naming";
4   import { logDebug, logError } from "./logger";
5
6   type Folder = GoogleAppsScript.Drive.Folder;
7
8   /**
9    * Anything that can hold folders: both `DriveApp` (the root) and a `Folder`
10   * expose `getFolders()` / `createFolder()`, so we accept either structurally.
11   */
12   interface FolderContainer {
13     getFolders(): GoogleAppsScript.Drive.FolderIterator;
14     createFolder(name: string): Folder;
15   }
16
17   export interface UpdateResult {
18     /** Number of files actually written to Drive in this call. */
19     addedFiles: number;
20     hasError: boolean;
21     /** Count of attachment-level errors in this call. */
22     errors: number;
23     /** Most recent error message, if any. */
24     lastError: string;
25     /** Gmail message ids fully handled in this call (for cross-run dedup). */
26     processedIds: string[];
27   }
28
29   /** Returns the named child folder, creating it if it does not exist yet. */
30   export function getOrCreateFolderInFolder(
31     parent: FolderContainer,
32     name: string,
33   ): Folder {
34     const folders = parent.getFolders();
35     while (folders.hasNext()) {
36       const folder = folders.next();
37       if (folder.getName() === name) {
38         logDebug("Found existing folder " + name);
39         return folder;

```

```

40     }
41 }
42 logDebug("Creating new folder " + name);
43 return parent.createFolder(name);
44 }
45
46 /**
47  * Resolves the destination sub-folder for an attachment, creating it under
48  * `head` and caching it in `existingFolders`. Returns null when the chosen
49  * structure yields no folder (caller falls back to the head folder).
50  */
51 function resolveFolder(
52     head: Folder,
53     existingFolders: Map<string, Folder>,
54     structure: string,
55     contentType: string,
56     date: Date,
57 ): Folder | null {
58     const name = getFolderNameForStructure(structure, contentType, date);
59     if (name === "") {
60         return null;
61     }
62     const cached = existingFolders.get(name);
63     if (cached) {
64         return cached;
65     }
66     const created = head.createFolder(name);
67     existingFolders.set(name, created);
68     return created;
69 }
70
71 /**
72  * Copies every allowed, not-yet-present attachment from the given threads into
73  * the GmailAttachments folder, organised per the user's chosen structure.
74  * Stops once `maxToAdd` files have been written.
75  *
76  * `alreadyProcessed` is the set of Gmail message ids handled in previous runs;
77  * such messages are skipped wholesale, avoiding a Drive `getFilesByName` probe
78  * per attachment when an overlapping date window is re-scanned. Ids fully
79  * handled here are returned in `processedIds` for the caller to persist.
80  */
81 export function updateDrive(
82     threads: GoogleAppsScript.Gmail.GmailThread[],
83     structure: string,
84     maxToAdd: number,
85     alreadyProcessed: ReadonlySet<string> = new Set(),
86 ): UpdateResult {
87     const head = getOrCreateFolderInFolder(DriveApp, GMAIL_ATTACHMENT_FOLDER);
88
89     // Pre-load existing sub-folders so we reuse rather than duplicate them.
90     const existingFolders = new Map<string, Folder>();
91     const currentFolders = head.getFolders();
92     while (currentFolders.hasNext()) {
93         const folder = currentFolders.next();
94         existingFolders.set(folder.getName(), folder);
95     }
96     logDebug("Already present folders: " + existingFolders.size);
97
98     const addedFiles = new Map<string, Folder>();
99     const processedIds: string[] = [];
100     let addMore = maxToAdd > 0;
101     let hasError = false;
102     let errors = 0;
103     let lastError = "";
104

```

```

105     for (const thread of threads) {
106         if (!addMore) break;
107         for (const message of thread.getMessages()) {
108             if (!addMore) break;
109             const messageId = message.getId();
110             // Skip messages we fully handled on a previous run.
111             if (alreadyProcessed.has(messageId)) {
112                 logDebug("Skipping already-processed message " + messageId);
113                 continue;
114             }
115             let messageFailed = false;
116             const from = message.getFrom();
117             // GmailMessage#getDate is typed as the stripped Base.Date interface but is
118             // a real JS Date at runtime.
119             const date = message.getDate() as unknown as Date;
120             const attachments = message.getAttachments();
121             let lastHandled = -1;
122             for (let a = 0; a < attachments.length && addMore; a++) {
123                 const attachment = attachments[a];
124                 lastHandled = a;
125                 const contentType = attachment.getContentType();
126                 const fileName = getAttachmentName(attachment.getName(), from, date);
127
128                 if (!isAllowedContentType(contentType) || addedFiles.has(fileName)) {
129                     logDebug("Rejected file of type: " + contentType);
130                     continue;
131                 }
132
133                 let folder = resolveFolder(
134                     head,
135                     existingFolders,
136                     structure,
137                     contentType,
138                     date,
139                 );
140                 if (folder == null) {
141                     // Unknown structure: drop it in the top-level folder for recovery.
142                     logError("Could not resolve directory; using top-level folder.");
143                     folder = head;
144                 }
145
146                 try {
147                     // Skip if a file with this name already exists in the folder.
148                     if (folder.GetFilesByName(fileName).hasNext()) {
149                         continue;
150                     }
151                     const blob = attachment.copyBlob();
152                     blob.setName(fileName);
153                     const file = folder.createFile(blob);
154                     file.setDescription(
155                         "Created from a Gmail Attachment from " +
156                         from +
157                         " sent on date " +
158                         date,
159                     );
160                     addedFiles.set(fileName, folder);
161                     Utilities.sleep(100);
162                     if (addedFiles.size === maxToAdd) {
163                         addMore = false;
164                     }
165                 } catch (e) {
166                     logError("Attachment copy failed (message " + messageId + "): " + e);
167                     hasError = true;
168                     messageFailed = true;
169                     errors++;

```



```

170         lastError = String(e);
171         Utilities.sleep(1000);
172     }
173 }
174 // Mark the message done only if we handled all its attachments cleanly.
175 // If we stopped early on the daily cap (lastHandled short of the end) or
176 // hit a transient failure, leave it unmarked so the next run retries it.
177 const completed = !messageFailed && lastHandled === attachments.length - 1;
178 if (completed) {
179     processedIds.push(messageId);
180 }
181 }
182 }
183
184 return { addedFiles: addedFiles.size, hasError, errors, lastError, processedIds };
185 }
186

```

5. user (2026-06-10T16:26:08.576Z)

```

1  import {
2      collectAddedMessageIds,
3      isHistoryExpired,
4      nextHistoryId,
5      type HistoryListResponse,
6  } from "../historySync";
7  import { logDebug } from "../logger";
8
9  type GmailThread = GoogleAppsScript.Gmail.GmailThread;
10
11  /** A bounded number of history pages to walk in a single run. */
12  const MAX_HISTORY_PAGES = 20;
13
14  /** The mailbox's current historyId, used to seed incremental sync. */
15  export function getCurrentHistoryId(): string | null {
16      const profile = Gmail.Users.getProfile("me");
17      return profile.historyId ?? null;
18  }
19
20  /** What an incremental scan yields: new message ids and the id to store next. */
21  export interface IncrementalChanges {
22      messageIds: string[];
23      historyId: string;
24  }
25
26  /**
27   * Lists messages added since `startHistoryId` via the Gmail advanced service,
28   * paging through results. Returns null when the history has expired (the caller
29   * must fall back to a full sync). Only `messageAdded` events are requested.
30   */
31  export function listAddedSince(
32      startHistoryId: string,
33  ): IncrementalChanges | null {
34      const responses: HistoryListResponse[] = [];
35      let pageToken: string | undefined;
36      try {
37          for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
38              const response = Gmail.Users.History.list("me", {
39                  startHistoryId,
40                  historyTypes: ["messageAdded"],
41                  pageToken,
42              }) as HistoryListResponse;
43              responses.push(response);
44              if (!response.nextPageToken) {
45                  break;
46              }
47          }
48      } catch (e) {
49          logDebug(e);
50      }
51      if (responses.length > 0) {
52          const messageIds = responses.flatMap((r) => r.messageIds);
53          const historyId = responses[responses.length - 1].historyId;
54          return { messageIds, historyId };
55      }
56      return null;
57  }
58

```

```

46         }
47         pageToken = response.nextPageToken;
48     }
49     } catch (e) {
50         if (isHistoryExpired(e)) {
51             logDebug("History id expired; full sync required.");
52             return null;
53         }
54         throw e;
55     }
56     return {
57         messageIds: collectAddedMessageIds(responses),
58         historyId: nextHistoryId(responses, startHistoryId),
59     };
60 }
61
62 /**
63  * Resolves the distinct Gmail threads containing the given message ids. Threads
64  * are de-duplicated by id so a multi-message thread is processed once. Messages
65  * that can no longer be fetched (deleted since the change was recorded) are
66  * skipped.
67  */
68 export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
69     const threads = new Map<string, GmailThread>();
70     for (const id of messageIds) {
71         const message = GmailApp.getMessageById(id);
72         if (!message) {
73             continue;
74         }
75         const thread = message.getThread();
76         threads.set(thread.getId(), thread);
77     }
78     return Array.from(threads.values());
79 }
80

```

6. user (2026-06-10T16:26:26.978Z)

```

1     /*
2     * The MIT License (MIT)
3     * Copyright (c) 2013 Avi Dullu
4     */
5
6     /** Top-level Drive folder that holds every synced attachment. */
7     export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";
8
9     /** Public project / support page shown in the setup UI. */
10    export const SUPPORT_URL =
11        "https://github.com/avidullu/gmail-attachments-to-gdrive";
12
13    /** Privacy policy shown in the setup UI (and the OAuth consent screen). */
14    export const PRIVACY_POLICY_URL =
15        "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";
16
17    /**
18     * Apps Script enforces a daily quota on file creation. We cap ourselves below
19     * it so a single run can never exhaust the account's allowance.
20     */
21    export const MAX_FILES_ADD_PER_DAY = 240;
22
23    // Time constants, in milliseconds.
24    export const ONE_MINUTE = 60_000;
25    export const ONE_DAY = 86_400_000;
26    export const THIRTY_DAYS = 2_592_000_000;
27    export const ONE_YEAR = 31_622_400_000;

```

```

28     export const SIX_HOURS = 21_600_000;
29
30     /**
31      * Wall-clock budget for a single sync execution. Apps Script kills consumer
32      * scripts at 6 minutes, so we stop at 5 to leave margin for the final
33      * bookkeeping and trigger scheduling.
34      */
35     export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;
36
37     /** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
38     export const MAX_SYNC_TRIGGERS = 12;
39
40     // Keys used in PropertiesService.getUserProperties().
41     export const PROP_FOLDER = "f";
42     export const PROP_LAST_SYNCED = "l";
43     export const PROP_FILES_ADDED_TODAY = "fa";
44     export const PROP_DEBUG = "dbg";
45     export const PROP_LAST_ERROR = "err";
46     export const PROP_PROCESSED_IDS = "pid";
47     export const PROP_RUN_STATS = "stats";
48     export const PROP_SUMMARY_LAST_SENT = "sumsent";
49     export const PROP_HISTORY_ID = "hid";
50
51     /** Flip to true to emit verbose Logger output during development. */
52     export const ENABLE_DEBUGGING = false;
53
54     /**
55      * Hybrid incremental sync. OFF by default so the proven date-window backfill
56      * remains the sole code path; flip to true to roll out the cheaper Gmail
57      * `history.list` steady state. When on, once a backfill catches up the current
58      * mailbox `historyId` is captured and subsequent runs process only newly-added
59      * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
60      * Requires the Gmail advanced service (declared in appsscript.json); no extra
61      * OAuth scope ? `gmail.readonly` already covers history reads.
62      */
63     export const ENABLE_INCREMENTAL_SYNC = false;
64
65     /**
66      * Daily summary email. OFF by default ? enabling it requires the
67      * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68      * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69      * effective user's own address).
70      */
71     export const ENABLE_SUMMARY_EMAIL = false;
72     export const SUMMARY_EMAIL_RECIPIENT = "";
73

```

7. user (2026-06-10T16:26:27.439Z)

```

1     import {
2         PROP_DEBUG,
3         PROP_FILES_ADDED_TODAY,
4         PROP_FOLDER,
5         PROP_HISTORY_ID,
6         PROP_LAST_ERROR,
7         PROP_LAST_SYNCED,
8         PROP_PROCESSED_IDS,
9         PROP_RUN_STATS,
10        PROP_SUMMARY_LAST_SENT,
11    } from "./config";
12    import { parseProcessedIds } from "./processed";
13    import { parseStats, parseStatsRaw, type RunStats } from "./stats";
14    import type { FilesAddedToday } from "./quota";
15
16    /**

```

```

17     * Thin wrappers over PropertiesService. All persistent sync state lives in the
18     * user properties store. Replaces the removed `UserProperties` global and the
19     * deprecated `Utilities.jsonParse/jsonStringify` helpers.
20     */
21     function props(): GoogleAppsScript.Properties.Properties {
22         return PropertiesService.getUserProperties();
23     }
24
25     export function getFolderStructure(): string {
26         return props().getProperty(PROP_FOLDER) ?? "";
27     }
28
29     export function setFolderStructure(structure: string): void {
30         props().setProperty(PROP_FOLDER, structure);
31     }
32
33     export function getLastSynced(): string {
34         return props().getProperty(PROP_LAST_SYNCED) ?? "";
35     }
36
37     export function setLastSynced(iso: string): void {
38         props().setProperty(PROP_LAST_SYNCED, iso);
39     }
40
41     export function getDebug(): string | null {
42         return props().getProperty(PROP_DEBUG);
43     }
44
45     export function setDebug(value: string): void {
46         props().setProperty(PROP_DEBUG, value);
47     }
48
49     export function getFilesAddedToday(): FilesAddedToday {
50         const raw = props().getProperty(PROP_FILES_ADDED_TODAY);
51         if (!raw) {
52             return { added: 0, date: new Date().toJSON() };
53         }
54         return JSON.parse(raw) as FilesAddedToday;
55     }
56
57     export function setFilesAddedToday(value: FilesAddedToday): void {
58         props().setProperty(PROP_FILES_ADDED_TODAY, JSON.stringify(value));
59     }
60
61     /** The set of Gmail message ids we have already processed. */
62     export function getProcessedIds(): string[] {
63         return parseProcessedIds(props().getProperty(PROP_PROCESSED_IDS));
64     }
65
66     export function setProcessedIds(ids: string[]): void {
67         props().setProperty(PROP_PROCESSED_IDS, JSON.stringify(ids));
68     }
69
70     /** Per-day run statistics accumulator (fresh for `day` if absent/stale). */
71     export function getRunStats(day: string): RunStats {
72         return parseStats(props().getProperty(PROP_RUN_STATS), day);
73     }
74
75     /** The persisted run stats as-is (may belong to a prior day), or null. */
76     export function getRunStatsRaw(): RunStats | null {
77         return parseStatsRaw(props().getProperty(PROP_RUN_STATS));
78     }
79
80     export function setRunStats(stats: RunStats): void {
81         props().setProperty(PROP_RUN_STATS, JSON.stringify(stats));

```

```

82     }
83
84     /** The day key (YYYY-MM-DD) of the last summary we sent, or null. */
85     export function getSummaryLastSent(): string | null {
86         return props().getProperty(PROP_SUMMARY_LAST_SENT);
87     }
88
89     export function setSummaryLastSent(day: string): void {
90         props().setProperty(PROP_SUMMARY_LAST_SENT, day);
91     }
92
93     /**
94      * The Gmail `historyId` from which incremental sync should resume, or null when
95      * we are still backfilling (date-window mode) or the id has expired.
96      */
97     export function getHistoryId(): string | null {
98         return props().getProperty(PROP_HISTORY_ID);
99     }
100
101     export function setHistoryId(id: string): void {
102         props().setProperty(PROP_HISTORY_ID, id);
103     }
104
105     /** Drops the stored historyId, forcing the next run back into backfill mode. */
106     export function clearHistoryId(): void {
107         props().deleteProperty(PROP_HISTORY_ID);
108     }
109
110     /** Records the last execution error so it can be inspected later. */
111     export function setLastError(message: string): void {
112         props().setProperty(
113             PROP_LAST_ERROR,
114             JSON.stringify({ message, at: new Date().toJSON() }),
115         );
116     }
117
118     /** Wipes all persisted state. Drive files are unaffected. */
119     export function clearAll(): void {
120         props().deleteAllProperties();
121     }
122

```

8. user (2026-06-10T16:26:33.738Z)

```

1     // Enable the opt-in incremental path for this suite only.
2     jest.mock("../src/config", () => ({
3         ...jest.requireActual("../src/config"),
4         ENABLE_INCREMENTAL_SYNC: true,
5     }));
6
7     import {
8         ONE_DAY,
9         PROP_DEBUG,
10        PROP_FILES_ADDED_TODAY,
11        PROP_FOLDER,
12        PROP_HISTORY_ID,
13        PROP_LAST_SYNCED,
14        PROP_PROCESSED_IDS,
15    } from "../src/config";
16    import { syncUserDrives } from "../src/sync";
17
18    type Store = Record<string, string>;
19    const g = globalThis as unknown as Record<string, unknown>;
20    const nowIso = () => new Date().toJSON();
21

```

```

22 function fakeProps(initial: Store) {
23     const store: Store = { ...initial };
24     return {
25         store,
26         getProperty: (k: string) => (k in store ? store[k] : null),
27         setProperty: (k: string, v: string) => {
28             store[k] = String(v);
29         },
30         deleteProperty: (k: string) => {
31             delete store[k];
32         },
33         deleteAllProperties: () => {
34             for (const k of Object.keys(store)) delete store[k];
35         },
36     };
37 }
38
39 function fakeScriptApp() {
40     const triggers: { getHandlerFunction: () => string }[] = [];
41     return {
42         newTrigger: (handler: string) => ({
43             timeBased: () => ({
44                 after: () => ({
45                     create: () => triggers.push({ getHandlerFunction: () => handler }),
46                 }),
47             }),
48         }),
49         getProjectTriggers: () => [...triggers],
50         deleteTrigger: () => undefined,
51     };
52 }
53
54 function iter<T>(items: T[]) {
55     let i = 0;
56     return { hasNext: () => i < items.length, next: () => items[i++] };
57 }
58
59 function installBase(props: ReturnType<typeof fakeProps>) {
60     g.PropertiesService = { getUserProperties: () => props };
61     g.ScriptApp = fakeScriptApp();
62     g.LockService = {
63         getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
64     };
65     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
66 }
67
68 function baseProps(overrides: Store = {}) {
69     return fakeProps({
70         [PROP_DEBUG]: "1",
71         [PROP_FOLDER]: "byContentType",
72         [PROP_LAST_SYNCED]: nowIso(),
73         [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
74         ...overrides,
75     });
76 }
77
78 describe("syncUserDrives ? incremental mode", () => {
79     it("processes messages added since the stored historyId and advances it", () => {
80         const props = baseProps({ [PROP_HISTORY_ID]: "100" });
81         installBase(props);
82
83         // A Drive head folder that accepts one file.
84         const head = {
85             getName: () => "GmailAttachments",
86             getFolders: () => iter([]),

```

```

87         createFolder: () => head,
88         getFilesByName: () => iter([]),
89         createFile: () => ({ setDescription: () => undefined }),
90     };
91     g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
92
93     const message = {
94         getId: () => "m1",
95         getFrom: () => "alice@example.com",
96         getDate: () => new Date(2012, 5, 6),
97         getAttachments: () => [
98             {
99                 getContentType: () => "application/pdf",
100                 getName: () => "report.pdf",
101                 copyBlob: () => ({ setName: () => undefined }),
102             },
103         ],
104     };
105     (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
106         () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
107     })),
108     g.Gmail = {
109         Users: {
110             getProfile: jest.fn(),
111             History: {
112                 list: jest.fn(() => ({
113                     history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
114                     historyId: "200",
115                 })),
116             },
117         },
118     };
119
120     syncUserDrives();
121
122     // Never falls back to a date-window search while the history is valid.
123     expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
124     expect(props.store[PROP_HISTORY_ID]).toBe("200");
125     expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
126 });
127
128 it("clears the cursor and rewinds lastSynced when the history expired", () => {
129     const props = baseProps({ [PROP_HISTORY_ID]: "1" });
130     installBase(props);
131     g.Gmail = {
132         Users: {
133             getProfile: jest.fn(),
134             History: {
135                 list: jest.fn(() => {
136                     throw new Error("call failed with error: 404 not found");
137                 }),
138             },
139         },
140     };
141
142     syncUserDrives();
143
144     expect(PROP_HISTORY_ID in props.store).toBe(false);
145     // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
146     const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
147     expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
148     // No history ? no incremental processing this run; backfill resumes next run.
149     expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
150 });

```

```

151
152     it("captures the mailbox historyId once backfill catches up", () => {
153         // No stored historyId ? backfill mode; lastSynced=now means we are already
154         // caught up, so the run captures the current historyId for next time.
155         const props = baseProps();
156         installBase(props);
157         g.Gmail = {
158             Users: {
159                 getProfile: jest.fn(() => ({ historyId: "555" })),
160                 History: { list: jest.fn() },
161             },
162         };
163
164         syncUserDrives();
165
166         expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
167         expect(props.store[PROP_HISTORY_ID]).toBe("555");
168     });
169 });
170

```

9. user (2026-06-10T16:26:34.231Z)

```

1      /*
2      * Pure helpers for the Gmail `users.history.list` incremental sync. These
3      * operate on plain data shapes (the JSON the Gmail advanced service returns)
4      * and have no GAS dependencies, so they are exercised directly in unit tests.
5      * The actual advanced-service calls live in the `gmailHistory` shell.
6      *
7      * See https://developers.google.com/workspace/gmail/api/guides/sync ? a stored
8      * `startHistoryId` returns only the changes since that point (~2 quota units vs
9      * ~5 for a list call). The id can expire, in which case the API responds 404
10     * and the caller must fall back to a full (date-window) sync.
11     */
12
13     /** A message stub as it appears inside a history record. */
14     export interface HistoryMessage {
15         id?: string;
16         threadId?: string;
17     }
18
19     /** One change record; we only care about messages added to the mailbox. */
20     export interface HistoryRecord {
21         messagesAdded?: { message?: HistoryMessage }[];
22     }
23
24     /** The shape of a `Gmail.Users.History.list` response we depend on. */
25     export interface HistoryListResponse {
26         history?: HistoryRecord[];
27         historyId?: string;
28         nextPageToken?: string;
29     }
30
31     /**
32     * Collects the unique ids of messages added across the given history pages, in
33     * first-seen order. Records without a `messagesAdded` entry (e.g. label-only
34     * changes) contribute nothing.
35     */
36     export function collectAddedMessageIds(
37         responses: HistoryListResponse[],
38     ): string[] {
39         const seen = new Set<string>();
40         const ids: string[] = [];
41         for (const response of responses) {
42             for (const record of response.history ?? []) {

```



```

43         for (const added of record.messagesAdded ?? []) {
44             const id = added.message?.id;
45             if (id && !seen.has(id)) {
46                 seen.add(id);
47                 ids.push(id);
48             }
49         }
50     }
51 }
52 return ids;
53 }
54
55 /**
56  * The newest `historyId` to persist for the next run: the last page's id, or
57  * the supplied fallback (the start id) if no page reported one.
58  */
59 export function nextHistoryId(
60     responses: HistoryListResponse[],
61     fallback: string,
62 ): string {
63     for (let i = responses.length - 1; i >= 0; i--) {
64         const id = responses[i].historyId;
65         if (id) {
66             return id;
67         }
68     }
69     return fallback;
70 }
71
72 /**
73  * Whether an error thrown by `history.list` means the stored `startHistoryId`
74  * has expired. Gmail responds with HTTP 404 ("Requested entity was not found")
75  * once the id predates the mailbox's retained history; the caller must then do
76  * a full sync. We match defensively on both the status code and the message
77  * text since the advanced service surfaces errors as a stringified exception.
78  */
79 export function isHistoryExpired(error: unknown): boolean {
80     const e = error as { message?: string; details?: { code?: number } } | null;
81     if (e && e.details && e.details.code === 404) {
82         return true;
83     }
84     const text = String(e?.message ?? error ?? "").toLowerCase();
85     return text.includes("404") || text.includes("not found");
86 }
87

```

10. user (2026-06-10T16:27:56.435Z)

Structured output provided successfully

11. assistant (2026-06-10T16:28:02.212Z)

Verification complete.